



Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Фундаментальные Науки»

КАФЕДРА

ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

Сжатое хранение разреженных матриц

Студент:

Подобедов В. В.

дата, подпись

Ф.И.О.

Преподаватель:

Волкова Л. Л.

дата, подпись

Ф.И.О.

Москва, 2023

Содержание

Введение	3
1 Аналитическая часть	4
2 Конструкторская часть	5
2.1 Схема Дженнингса	5
2.1.1 Описание схемы	5
2.1.2 Сжатие матрицы	5
2.1.3 Сложение разреженных матриц без распаковки	5
2.2 Кольцевая схема Рейнбольдта - Местеньи	6
2.2.1 Описание схемы	6
2.2.2 Сжатие матрицы	6
2.2.3 Сложение разреженных матриц без распаковки	6
2.2.4 Умножение разреженных матриц без распаковки	7
3 Технологическая часть	8
4 Исследовательская часть	24
Заключение	34
Список используемых источников	35

Введение

Цель лабораторной работы: провести сравнительный анализ алгоритмов поиска редакционных расстояний.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать две схемы сжатого хранения разреженных матриц — Дженнинга и по варианту: кольцевая схема или разреженный строчный формат.
2. Разработать алгоритмы сжатия, распаковки, сложения матриц для обеих схем и алгоритм умножения матриц для схемы по варианту.
3. Реализовать разработанные алгоритмы.
4. Выполнить тестирование реализации разработанных алгоритмов.
5. Оценить эффективность сжатого хранения матриц в обеих схемах для матрицы действительных чисел с 5–10 % ненулевых элементов.

Согласно варианту, требуется разработать следующие две схемы.

1. Схема Дженнинга. Сжатые матрицы нужно сложить, если это возможно, и распаковать, записав результат в файл.
2. Схема Рейнбольдта-Местеньи. Сложить и умножить сжатые матрицы, если это допускают размеры матриц; запаковать их и записать результаты в файл.

1 Аналитическая часть

Матрица — математический объект, записываемый в виде прямоугольной таблицы, который представляет собой совокупность строк и столбцов, на пересечении которых находятся его элементы. Количество строк и столбцов задает размер матрицы.

Разреженная матрица — матрица с 5–10 % ненулевых значений.

Схема Дженнингса (профильная схема хранения симметричных матриц) — это эффективный метод представления и хранения разреженных матриц, то есть матриц, в которых большинство элементов равны нулю. Эта схема разработана для оптимизации использования памяти и ускорения операций с разреженными матрицами. Все ненулевые элементы, которые находятся не выше главной диагонали с сохранением порядка хранятся в матрице и, возможно, некоторые нули, если они необходимы для сохранения порядка элементов.

Кольцевая схема (Рейнбольта — Местеньи) — это метод представления разреженных матриц, который удобен для хранения данных в виде кольцевых структур, минимизируя количество информации, необходимой для описания ненулевых элементов матрицы.

2 Конструкторская часть

2.1 Схема Дженнингса

2.1.1 Описание схемы

Схема Дженнингса [1] — это метод для хранения симметричных матриц и эффективного выполнения операций над ними. Разреженные матрицы имеют много нулей, и схема Дженнингса позволяет минимизировать использование памяти и ускорить операции с такими матрицами. Основная идея заключается в создании двух массивов: AN — массив всех ненулевых элементов, находящихся не выше главной диагонали матрицы и, возможно, служебных нулей (служебным нулём считается такой элемент, который находится либо на главной диагонали, либо в строке между ненулевым элементом и элементом с главной диагонали) и D — массив индексов элементов из AN , лежащих на диагонали матрицы.

2.1.2 Сжатие матрицы

Для сжатия матрицы по схеме Дженнингса необходимо использовать вышеперечисленные массивы AN и D . Для успешного выполнения алгоритма необходимо идти по строкам матрицы и записывать все ненулевые элементы матрицы, находящиеся под главной диагональю. Если был найден первый ненулевой элемент в строке, то записываются все последующие элементы, включая диагональный элемент этой строки. Например, если рассмотреть строку 0, 7, 0, 8, 9, где 9 — элемент главной диагонали, то в AN запишутся числа 7, 0, 8, 9. Далее, считывается индекс в массиве AN , который принадлежит диагональному элементу и добавляется в массив D . Курсор смещается на следующую строку, и осуществляется следующий шаг цикла по строкам.

2.1.3 Сложение разреженных матриц без распаковки

Для сложения разреженных матриц нужно сначала сложить самый первый диагональный элемент. Далее нужно рассмотреть разность индексов соседних диагональных элементов. Пользуясь этим фактом, можно заполнить матрицу AN и D для суммы матриц. Если разность индексов соседних элементов в первой матрице больше, чем во второй, то нужно добавлять элементы первой матрицы в AN суммы до того момента, пока разность не станет равной. После того, как разность стала равной нужно складывать элементы и проверять, чтобы сумма не равнялась нулю и добавлять в матрицу суммы AN . Когда курсор дойдет до диагонального элемента, то считывается индекс из первого массива D и добавляется в массив D суммы. Аналогично работает если разность индексов соседних элементов во второй матрице больше, чем в первой. Если же разность одинаковая для двух матриц, то элементы суммируются, проверяются на равенство нулю и добавляются в массивы AN и D .

2.2 Кольцевая схема Рейнбольдта - Местеньи

2.2.1 Описание схемы

Кольцевая схема Рейнбольдта — Местеньи [1] — это метод хранения произвольных разреженных матриц. Разреженные матрицы имеют много нулей, и алгоритм Дженнингса позволяет минимизировать использование памяти и ускорить операции с такими матрицами. Основная идея заключается в создании пяти массивов. AN — массив всех ненулевых элементов матрицы. NR — индекс (по AN) следующего ненулевого элемента той же строки. Последний элемент строки смотрит на начальный элемент строки. NC — индекс (по AN) следующего ненулевого элемента того же столбца. Последний элемент столбца смотрит на начальный элемент столбца. JR — индекс (по AN) начальных элементов строк. JC — индекс (по AN) начальных элементов столбцов.

2.2.2 Сжатие матрицы

Для сжатия матрицы используя кольцевую схему необходимо считывать поэлементно матрицу, делать проверку на ненулевой элемент и выполнять следующий алгоритм. Если элемент является самым первым в строке i , то изменить $JR[i]$ с -1 на значение счётчика, который был создан в начале программы и добавить ненулевой элемент в конец массива NR . Аналогично обновляется массив JC . Если элемент является самым первым в столбце j , то меняется $JC[j]$ с -1 на значение счётчика. Если элемент не является первым в строке, то необходимо идти курсором по массиву NR , пока не будет найден элемент, который ссылается на первый элемент строки. Его значение обновить на значение счётчика, а в конец массива добавить индекс первого элемента строки. Аналогичные действия нужно совершить для заполнения массивов NC и JC .

2.2.3 Сложение разреженных матриц без распаковки

В начале выполнения этого действия необходимо проверить сложимы ли матрицы. Если матрицы разных размеров, то сложение не возможно по определению. Если же матрицы одинаковых размеров, то необходимо создать три счётчика для каждого из массивов AN_1 , AN_2 , AN_sum . Для каждого из складываемых массивов найти строчную и столбцовую координату. Если один из элементов находится в строке, которая выше строки другого элемента, то добавить элемент из верхней строки в массив AN_sum и прибавить к его счётчику единицу. Если элементы находятся в одинаковых строках, то сравнить их столбцовые координаты. Если один из элементов имеет меньшую столбцовую координату, то добавить его в массив AN_sum и добавить единицу к его счётчику. Если столбцовые и строчные координаты равны, то сложить элементы, проверить их на равенство нулю и, если проверка пройдена, то добавить в массив AN_sum их сумму.

2.2.4 Умножение разреженных матриц без распаковки

При выполнении этого пункта необходимо учитывать, что умножение возможно тогда и только тогда, когда количество строк первой матрицы равно количеству столбцов второй матрицы или количество столбцов первой матрицы равно количеству строк второй матрицы. Если умножение возможно, то используя курсоры по строке одного массива и по столбцу второго массива умножить строку на столбец. Если произведение будет являться ненулевым, то добавить его в массив произведения AN , попутно заполняя другие массивы необходимые для хранения матрицы в кольцевой схеме.

3 Технологическая часть

Лабораторная работа была реализована на языке программирования C++. Кроме стандартной библиотеки `<iostream>`, которая предоставляет возможность пользоваться средствами ввода-вывода для стандартной консоли, была подключена библиотека `<fstream>`. Она включает в себя набор классов, методов и функций, которые предоставляют интерфейс для чтения/записи данных из/в файл. Также для удобства чтения матрицы из файла был подключена библиотека `<sstream>`, которая нужна для организации работы со строками, через интерфейс потоков.

Функции, разработанные в ходе написания программы, следующие.

- Функция `“output_krm”` — функция, которая считывает матрицу из файла.
- Функция `“djennings”` — функция, которая выполняет алгоритм сжатия матрицы по схеме Дженнингса.
- Функция `“print_matrix”` — функция, которая печатает матрицу в консоль.
- Функция `“inputt_file”` — функция, которая записывает матрицу после всех преобразований в файл.
- Функция `“compression_krm ”` — функция, которая выполняет алгоритм сжатия матрицы по кольцевой схеме.
- Функция `“summ_krm”` — функция, которая складывает две матрицы для матриц из кольцевой схемы.
- Функция `“multiplic_krm”` — функция, которая умножает две матрицы из кольцевой схемы.
- Функция `“num_col_el”` — функция, которая выводит столбец в котором находится элемент.
- Функция `“num_row_el”` — функция, которая выводит номер строки в котром находится элемент.
- Функция `“unpack_ring_RM”` — функция, которая составляет матрицу, пользуясь вспомогательными массивами.
- Функция `“print_arrow”` — функция, которая выводит массив.

В программе реализовано переключение между двумя режимами. Пользователь может самостоятельно выбрать каким алгоритмом он хочет пользоваться. При введении цифры 1 пользователю становится доступна схема Дженнингса. При вводе цифры 2 пользователь имеет возможность доступа к кольцевой схеме.

Далее приведен листинг реализации алгоритмов (листинг 1).

Листинг 1 – Исходный код программы для работы со схемой Дженнингса или с кольцевой схемой

```
#include <iostream>
#include <string>
#include <fstream>
#include <vector>
#include <sstream>

using namespace std;

void output_krm(vector<vector<int>>& matr, string txt, int &row, int &col
) {
    string filename = "C:\\Users\\Kononova\\Desktop\\мгту\\имНЭБаумана
        ..\\3 семестр\\Типы\\структуры данных\\лаб 2\\_2\\" + txt + ".txt";
    ifstream inputFile(filename);
    if (!inputFile.is_open()) {
        cout << "Не удалось открыть файл " << endl;
    }

    int rows_1 = 0;
    int cols_1 = 0;
    string line_1;
    while (getline(inputFile, line_1)) {
        vector<int> row_1;
        istringstream iss(line_1);
        int value;

        while (iss >> value) {
            row_1.push_back(value);
        }

        if (row_1.size() > 0) {
            matr.push_back(row_1);

            if (row_1.size() > cols_1) {
                cols_1 = row_1.size();
            }

            ++rows_1;
        }
    }
    row = rows_1;
}
```

```

        col = cols_1;
        inputFile.close();
    }

void djennings(vector<vector<int>>&matrix, vector<int> &AN, vector<int> &
    D) {
    for (int i = 0; i < matrix.size(); i++) {
        bool flagg = false;
        for (int j = 0; j <= i; j++) {
            if (matrix[i][j] != 0) {
                flagg = true;
            }
            if (flagg == true || i == j) {
                AN.push_back(matrix[i][j]);
                if (i == j) D.push_back(AN.size());
            }
        }
    }
}

void print_matrix(vector<vector<int>>& matrix) {
    for (const auto &row : matrix) {
        for (int num : row) {
            cout << num << " ";
        }
        cout << endl;
    }
    cout << endl;
}

void inputt_file(const vector<vector<int>>& matr, string txt) {
    string filename = "C:\\Users\\Kononova\\Desktop\\input" + txt +
        ".txt";
    ofstream outputFile(filename);

    if (outputFile.is_open()) {
        for (const vector<int>& row : matr) {
            for (int value : row) {
                outputFile << value << ' ';
            }
            outputFile << '\n';
        }
    }
}

```

```

        outputFile.close();
        cout << "Данные успешно записаны в файл      'input" + txt + ".
              txt'" << std::endl;
    }
    else {
        cerr << "Ошибка открытия файла для записи      " << std::endl;
    }
}

void compression_krm(vector<vector<int>>&matrix, vector<int> &AN_krm,
    vector<int> &NR_krm, vector<int> &NC_krm, vector<int> &JR_krm, vector<
int> &JC_krm, int &row, int &col) {
    int counter = 0;
    for (int i = 0; i < row; i++) {
        for (int j = 0; j < col; j++) {
            if (matrix[i][j] != 0) {
                if (JR_krm[i] == -1) {
                    JR_krm[i] = counter;
                    NR_krm.push_back(counter);
                }
                else {
                    int k = JR_krm[i];
                    while (NR_krm[k] != JR_krm[i]) {
                        k = NR_krm[k];
                    }
                    NR_krm[k] = counter;
                    NR_krm.push_back(JR_krm[i]);
                }
            }
            if (JC_krm[j] == -1) {
                JC_krm[j] = counter;
                NC_krm.push_back(counter);
            }
            else {
                int k = JC_krm[j];
                while (NC_krm[k] != JC_krm[j]) {
                    k = NC_krm[k];
                }
                NC_krm[k] = counter;
                NC_krm.push_back(JC_krm[j]);
            }
            AN_krm.push_back(matrix[i][j]);
        }
    }
}

```

```

        counter++;
    }
}

}

int num_col_el(vector<int>&an, vector<int>&nc, vector<int>&jc, int coord)
{
    for (int i = 0; i < jc.size(); i++) {
        if (jc[i] == coord) {
            return i;
        }
    }
    while (true) {
        if (coord > nc.size() - 1) {
            return 1;
        }
        coord = nc[coord];
        for (int i = 0; i < jc.size(); i++) {
            if (jc[i] == coord) {
                return i;
            }
        }
    }
}

```

```

int num_row_el(vector<int>&an, vector<int>&nr, vector<int>&jr, int coord)
{
    for (int i = 0; i < jr.size(); i++) {
        if (jr[i] == coord) {
            return i;
        }
    }
    while (true) {
        if (coord > nr.size() - 1) {
            return 1;
        }
        coord = nr[coord];
        for (int i = 0; i < jr.size(); i++) {
            if (jr[i] == coord) {
                return i;
            }
        }
    }
}

```

```

        }
    }
}

void unpack_ring_RM(vector<vector<int>>&a, vector<int>&an, vector<int>&nr
    , vector<int>&nc, vector<int>&jr, vector<int>&jc) {
    for (int i = 0; i < an.size(); i++) {
        int row = num_row_el(an, nr, jr, i);
        int col = num_col_el(an, nc, jc, i);
        a[row][col] = an[i];
    }
}

void print_arrow(vector<int>&arr) {
    for (int i = 0; i < arr.size(); i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void summ_krm(vector<vector<int>>&matrix1, vector<vector<int>>&matrix2,
    vector<int> &AN_krm, vector<int> &NR_krm, vector<int> &NC_krm, vector<
    int> &JR_krm, vector<int> &JC_krm, int &row_1, int &col_1, int &row_2,
    int &col_2) {
    if (row_1 == row_2 && col_1 == col_2) {
        int counter = 0;
        for (int i = 0; i < row_1; i++) {
            for (int j = 0; j < col_1; j++) {
                if (matrix1[i][j] + matrix2[i][j] != 0) {
                    if (JR_krm[i] == -1) {
                        JR_krm[i] = counter;
                        NR_krm.push_back(counter)
                        ;
                    }
                    else {
                        int k = JR_krm[i];
                        while (NR_krm[k] !=
                            JR_krm[i]) {
                            k = NR_krm[k];
                        }
                        NR_krm[k] = counter;
                    }
                }
            }
        }
    }
}

```

```

        NR_krm.push_back(JR_krm[i
        ]);
    }
    if (JC_krm[j] == -1) {
        JC_krm[j] = counter;
        NC_krm.push_back(counter)
        ;
    }
    else {
        int k = JC_krm[j];
        while (NC_krm[k] !=
            JC_krm[j]) {
            k = NC_krm[k];
        }
        NC_krm[k] = counter;
        NC_krm.push_back(JC_krm[j
        ]);
    }
    AN_krm.push_back(matrix1[i][j] +
        matrix2[i][j]);
    counter++;
}

}

}

vector<vector<int>> matrix_krm_summ(row_1, vector<int>(<
    col_1));
unpack_ring_RM(matrix_krm_summ, AN_krm, NR_krm, NC_krm,
    JR_krm, JC_krm);
cout << "Матрица суммы" << endl;
print_matrix(matrix_krm_summ);
cout << "AN суммymатриц : ";
print_arrow(AN_krm);
cout << "NR суммymатриц : ";
print_arrow(NR_krm);
cout << "NC суммymатриц : ";
print_arrow(NC_krm);
cout << "JR суммymатриц : ";
print_arrow(JR_krm);
cout << "JC суммymатриц : ";
print_arrow(JC_krm);
cout << endl << endl;
inputt_file(matrix_krm_summ, "2");

```

```

    }
    else {
        cout << "Невозможно сложить" << endl;
    }
}

void multiplic_krm(vector<vector<int>>&matrix1, vector<vector<int>>&
matrix2, vector<int> &AN_krm, vector<int> &NR_krm, vector<int> &NC_krm
, vector<int> &JR_krm, vector<int> &JC_krm, int &row_1, int &col_1,
int &row_2, int &col_2) {
    if (row_1 == col_2 || row_2 == col_1) {
        vector<vector<int>>mult(row_1, vector<int>(col_2));
        for (int i = 0; i < row_1; i++) {
            for (int j = 0; j < col_2; j++) {
                for (int k = 0; k < col_1; k++) {
                    mult[i][j] += matrix1[i][k] *
                        matrix2[k][j];
                }
            }
        }
        compression_krm(mult, AN_krm, NR_krm, NC_krm, JR_krm,
            JC_krm, row_1, col_2);
        vector<vector<int>> matrix_krm(row_1, vector<int>(col_2))
            ;
        unpack_ring_RM(matrix_krm, AN_krm, NR_krm, NC_krm, JR_krm
            , JC_krm);
        cout << "Матрица произведения" << endl;
        print_matrix(matrix_krm);
        cout << "AN произведения матриц : ";
        print_arrow(AN_krm);
        cout << "NR произведения матриц : ";
        print_arrow(NR_krm);
        cout << "NC произведения матриц : ";
        print_arrow(NC_krm);
        cout << "JR произведения матриц : ";
        print_arrow(JR_krm);
        cout << "JC произведения матриц : ";
        print_arrow(JC_krm);
        inputt_file(matrix_krm, "3");
    }
    else {
        cout << "Умножение невозможно" << endl;
    }
}

```

```

    }
}

```

```

int main() {
    setlocale(0, "");
    int choose = 0;
    cout << "Алгоритм Дженнингса - 1, кольцевая схема - 2: ";
    cin >> choose;
    if (choose == 1) {
        vector<vector<int>> matrix_1;
        int row_1 = 0, col_1 = 0;
        output_krm(matrix_1, "matr_1", row_1, col_1);
        cout << "Первая матрица" << endl;
        print_matrix(matrix_1);
        vector<int> AN_1;
        vector<int> D_1;
        djennings(matrix_1, AN_1, D_1);
        cout << "AN первой матрицы : ";
        print_arrow(AN_1);
        cout << "D первой матрицы : ";
        print_arrow(D_1);
        cout << endl;

        vector<vector<int>> matrix_2;
        int row_2 = 0, col_2 = 0;
        output_krm(matrix_2, "matr_2", row_2, col_2);
        cout << "Вторая матрица" << endl;
        print_matrix(matrix_2);
        vector<int> AN_2;
        vector<int> D_2;
        djennings(matrix_2, AN_2, D_2);
        cout << "AN второй матрицы : ";
        print_arrow(AN_2);
        cout << "AN второй матрицы : ";
        print_arrow(D_2);
        cout << endl;

        vector<int> summ_matr;
        vector<int> D_summ_matr;
        if (D_1.size() == D_2.size()) {
            for (int i = 0; i < D_1.size(); i++) {

```

```

if (i == 0) {
    summ_matr.push_back(AN_1[i] +
        AN_2[i]);
    D_summ_matr.push_back(1);
}
else {
    int razn_1 = D_1[i] - D_1[i - 1];
    int razn_2 = D_2[i] - D_2[i - 1];
    if (razn_1 < razn_2) {
        for (int k = 0; k <
            razn_2 - razn_1; k++)
        {
            summ_matr.
                push_back(AN_2
                    [D_2[i - 1] +
                        k]);
        }
        if (razn_1 > 2) {
            for (int h = 0; h
                < razn_1 - 1;
                h++) {
                int check
                    =
                    AN_1[
                        D_1[i
                            - 1] +
                                h] +
                    AN_2[
                        D_1[i
                            - 1] +
                                h];
                if (check
                    != 0)

                    summ_matr
                        .
                        push_back
                            (check
                                );
            }
        }
    }
}
}

```

```

        if (razn_1 > razn_2) {
            for (int k = 0; k <
                razn_1 - razn_2; k++)
            {
                summ_matr.
                    push_back(AN_1
                        [D_1[i - 1] +
                            k]);
            }
            if (razn_2 > 2) {
                for (int h = 0; h
                    < razn_2 - 1;
                    h++) {
                    int check
                        =
                            AN_1[
                                D_2[i
                                    - 1] +
                                        h] +
                            AN_2[
                                D_2[i
                                    - 1] +
                                        h];
                    if (check
                        != 0)

                        summ_matr
                            .
                                push_back
                                    (check
                                        );
                }
            }
        }
        summ_matr.push_back(AN_1[D_1[i] -
            1] + AN_2[D_2[i] - 1]);
        D_summ_matr.push_back(summ_matr.
            size());
    }
}

vector<vector<int>> matrix(D_summ_matr.size(),
    vector<int>(D_summ_matr.size()));

```

```

        for (int i = 0; i < D_summ_matr.size(); i++) {
            matrix[i][i] = summ_matr[D_summ_matr[i] -
                1];
            if (i != 0) {
                int null_count = 0;
                int h = D_summ_matr[i - 1] + 1;
                while (h < D_summ_matr[i]) {
                    matrix[null_count][i] =
                        summ_matr[D_summ_matr[
                            i - 1] + null_count];
                    matrix[i][null_count] =
                        summ_matr[D_summ_matr[
                            i - 1] + null_count];
                    null_count++;
                    h++;
                }
            }
        }

        cout << "Сумма матриц: " << endl;
        print_matrix(matrix);
        cout << "AN суммymатриц : ";
        print_arrow(summ_matr);
        cout << "D суммymатриц ";
        print_arrow(D_summ_matr);
        cout << endl;
        inputt_file(matrix, "1");
        cout << endl;
    }
    else {
        cout << "Сложение матриц невозможно " << endl;
    }
}

else {
    vector<vector<int>> matrix_3;
    int row_3 = 0, col_3 = 0;
    output_krm(matrix_3, "test1", row_3, col_3);
    cout << "Первая матрица" << endl;
    print_matrix(matrix_3);
    vector<int> AN_krm_1;
    vector<int> NR_krm_1;
    vector<int> NC_krm_1;
    vector<int> JR_krm_1(row_3, -1);

```

```

vector<int> JC_krm_1(col_3, -1);
compression_krm(matrix_3, AN_krm_1, NR_krm_1, NC_krm_1,
    JR_krm_1, JC_krm_1, row_3, col_3);
cout << "AN первой матрицы : ";
print_arrow(AN_krm_1);
cout << "NR первой матрицы : ";
print_arrow(NR_krm_1);
cout << "NC первой матрицы : ";
print_arrow(NC_krm_1);
cout << "JR первой матрицы : ";
print_arrow(JR_krm_1);
cout << "JC первой матрицы : ";
print_arrow(JC_krm_1);
cout << endl << endl;

vector<vector<int>> matrix_4;
int row_4 = 0, col_4 = 0;
output_krm(matrix_4, "test2", row_4, col_4);
cout << "Вторая матрица" << endl;
print_matrix(matrix_4);
vector<int> AN_krm_2;
vector<int> NR_krm_2;
vector<int> NC_krm_2;
vector<int> JR_krm_2(row_4, -1);
vector<int> JC_krm_2(col_4, -1);
compression_krm(matrix_4, AN_krm_2, NR_krm_2, NC_krm_2,
    JR_krm_2, JC_krm_2, row_4, col_4);
cout << "AN второй матрицы : ";
print_arrow(AN_krm_2);
cout << "NR второй матрицы : ";
print_arrow(NR_krm_2);
cout << "NC второй матрицы : ";
print_arrow(NC_krm_2);
cout << "JR второй матрицы : ";
print_arrow(JR_krm_2);
cout << "JC второй матрицы : ";
print_arrow(JC_krm_2);
cout << endl << endl;

vector<int> AN_krm_sum;
vector<int> NR_krm_sum;
vector<int> NC_krm_sum;

```

```

vector<int> JR_krm_sum(row_3, -1);
vector<int> JC_krm_sum(col_3, -1);
summ_krm(matrix_3, matrix_4, AN_krm_sum, NR_krm_sum,
          NC_krm_sum, JR_krm_sum, JC_krm_sum, row_3, col_3,
          row_4, col_4);

vector<int> AN_krm_mult;
vector<int> NR_krm_mult;
vector<int> NC_krm_mult;
vector<int> JR_krm_mult(row_3, -1);
vector<int> JC_krm_mult(col_4, -1);
multiplic_krm(matrix_3, matrix_4, AN_krm_mult,
              NR_krm_mult, NC_krm_mult, JR_krm_mult, JC_krm_mult,
              row_3, col_3, row_4, col_4);

}
return 0;
}

```

Для проверки работоспособности программы были выполнены следующие тесты. Тесты на сложение матриц представлены в таблице 1

Таблица 1 – Тесты на сложение матриц

Описание теста	Входные данные	Ожидаемый результат
Сложение матриц с несовпадающими размерами	Две квадратные матрицы разных размеров: $a \times a$ и $b \times b$	Ошибка
Сложение нулевых матриц	Две нулевые матрицы одинакового размера $n \times n$	Нулевая матрица размера $n \times n$
Сложение нулевой матрицы M и ненулевой матрицы N	Матрица из нулевых элементов размера $n \times n$ и ненулевая матрица размера $n \times n$	Ненулевая матрица N
Сложение ненулевой матрицы M и нулевой матрицы N	Матрица из ненулевых элементов размера $n \times n$ и нулевая матрица размера $n \times n$	Ненулевая матрица M
Сложение двух ненулевых матриц одинакового размера	Ненулевые матрицы M и N	Результирующая ненулевая матрица

Пример сложения ненулевых матриц A и B размеров $n \times m$:

$$\begin{pmatrix} 1 & 0 & 7 & 0 \\ 0 & 2 & 0 & 8 \\ 7 & 0 & 3 & 6 \\ 0 & 8 & 6 & 4 \end{pmatrix} + \begin{pmatrix} 7 & 8 & 0 & 6 \\ 8 & 9 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 6 & 0 & 0 & 4 \end{pmatrix} = \begin{pmatrix} 8 & 8 & 7 & 6 \\ 8 & 11 & 0 & 8 \\ 7 & 0 & 6 & 6 \\ 6 & 8 & 6 & 8 \end{pmatrix}$$

Пример сложения нулевой матрицы А и ненулевой матрицы В размеров $n \times n$:

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} + \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix}$$

Для проверки работоспособности умножения были выполнены следующие тесты. Тесты на умножение матриц представлены в таблице 2

Таблица 2 – Тесты на умножение матриц

Описание теста	Входные данные	Ожидаемый результат
Умножение матриц с несовпадающими размерами	Две квадратные матрицы разных размеров: $a \times b$ и $c \times d$	Ошибка
Умножение нулевых матриц	Две нулевые матрицы разного размера $a \times b$ на $b \times c$	Нулевая матрица размера $a \times c$
Умножение нулевой матрицы М и ненулевой матрицы N	Матрица из нулевых элементов размера $a \times b$ и ненулевая матрица размера $b \times c$	Нулевая матрица размера $a \times c$
Умножение ненулевой матрицы М и нулевой матрицы N	Матрица из ненулевых элементов размера $a \times b$ и нулевая матрица размера $b \times c$	Нулевая матрица размера $a \times c$
Умножение двух ненулевых матриц	Ненулевые матрицы М размера $a \times b$ и N размера $b \times c$	Результирующая ненулевая матрица размера $a \times c$

Пример умножения ненулевых матриц А и В размеров $n \times m$:

$$\begin{pmatrix} 2 & 0 & 5 & 4 \\ -5 & 0 & 9 & 0 \\ 0 & 0 & 0 & 3 \\ 1 & 0 & 7 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 & 0 & 0 & 5 \\ 0 & 7 & 8 & 0 & 10 \\ 0 & 12 & 0 & 0 & 0 \\ 0 & 17 & 0 & 19 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 132 & 0 & 76 & 10 \\ -5 & 98 & 0 & 0 & -25 \\ 0 & 51 & 0 & 57 & 0 \\ 1 & 86 & 0 & 0 & 5 \\ 0 & 12 & 0 & 0 & 0 \end{pmatrix}$$

Пример умножения нулевой матрицы А и ненулевой матрицы В размеров $n \times m$ и $m \times p$ соответственно:

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 & 3 & 4 & 17 \\ 5 & 6 & 7 & 8 & 18 \\ 9 & 10 & 11 & 12 & 19 \\ 13 & 14 & 15 & 16 & 20 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Все тесты были пройдены успешно.

4 Исследовательская часть

Далее на рисунках представлены промежуточные результаты программы и входные файлы и выходной файл, сформированный программой.

Результат вычисления суммы матриц разного размера по схеме Дженнинга приведен на рис. 1.

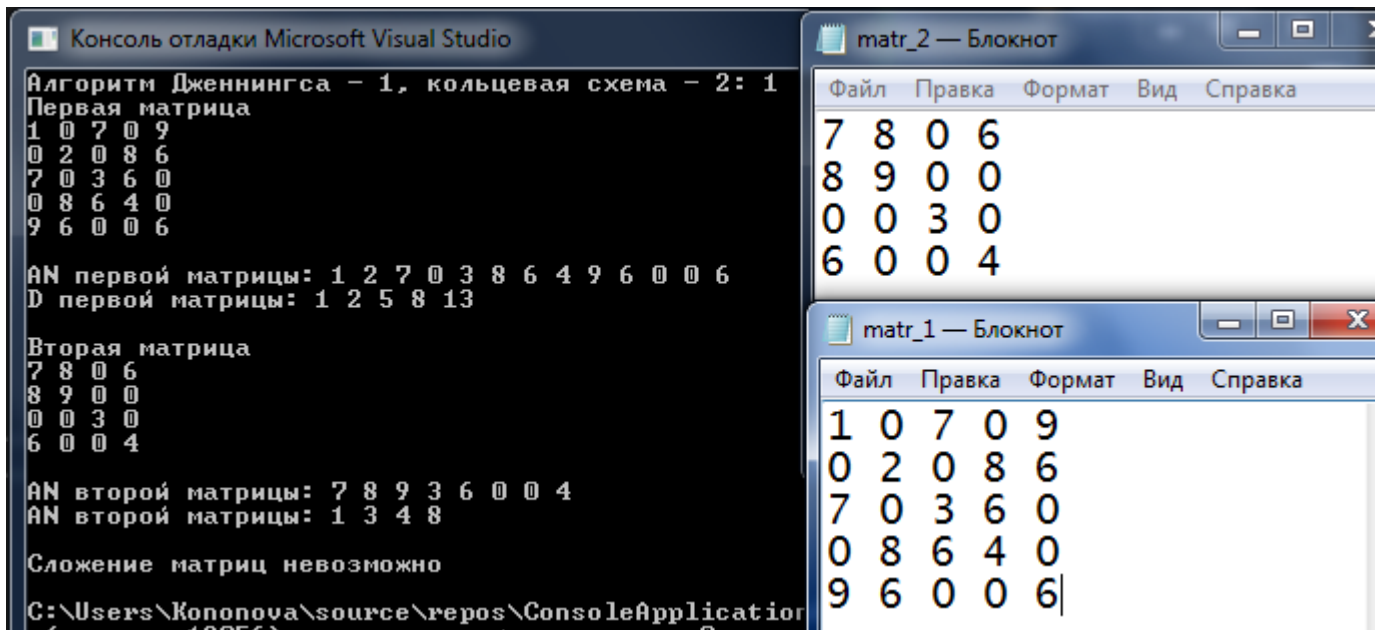
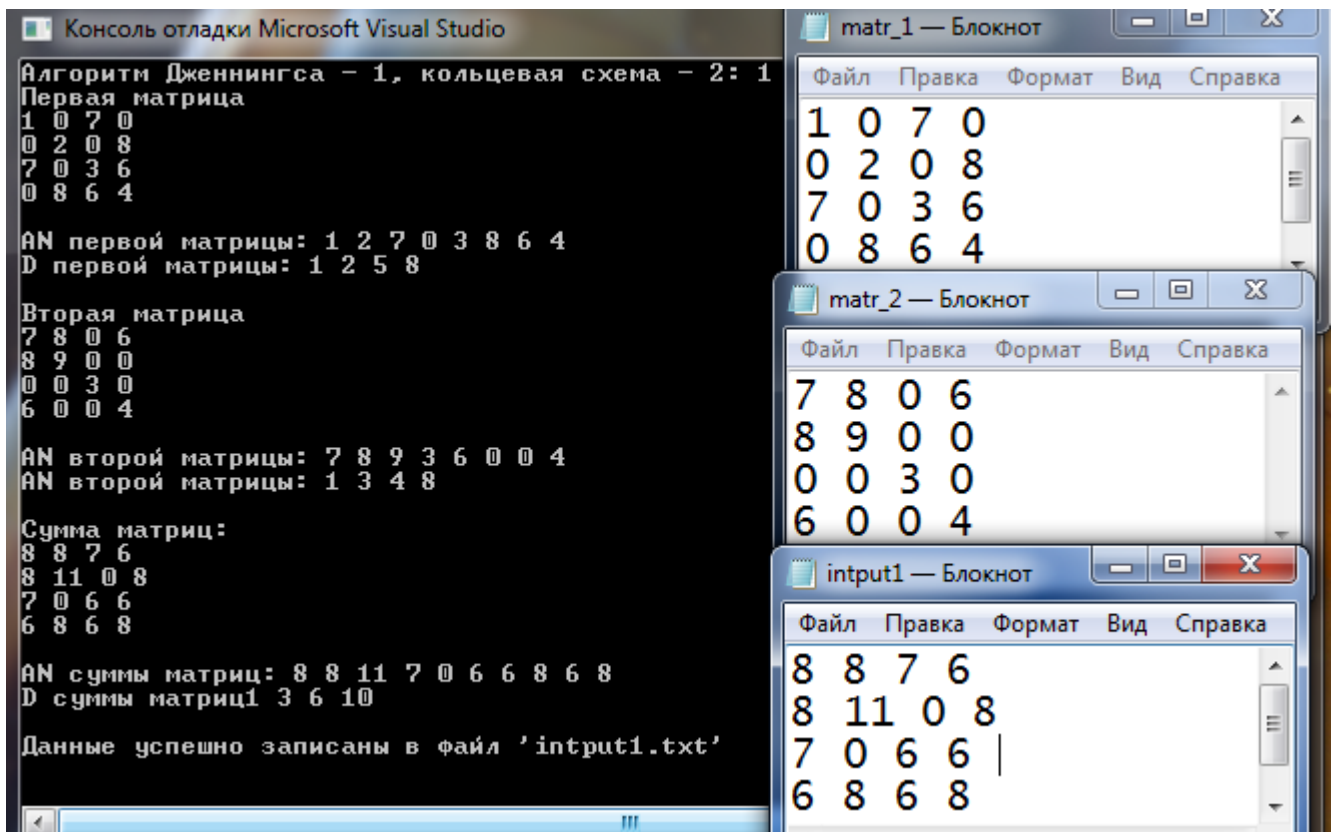


Рис. 1 – Работоспособность программы сложения по схеме Дженнинга

Результат вычисления суммы различных матриц одинакового размера по схеме Дженнинга приведен на рис. 2.



```
Консоль отладки Microsoft Visual Studio
Алгоритм Дженнинга - 1, кольцевая схема - 2: 1
Первая матрица
1 0 7 0
0 2 0 8
7 0 3 6
0 8 6 4

AN первой матрицы: 1 2 7 0 3 8 6 4
D первой матрицы: 1 2 5 8

Вторая матрица
7 8 0 6
8 9 0 0
0 0 3 0
6 0 0 4

AN второй матрицы: 7 8 9 3 6 0 0 4
D второй матрицы: 1 3 4 8

Сумма матриц:
8 8 7 6
8 11 0 8
7 0 6 6
6 8 6 8

AN суммы матриц: 8 8 11 7 0 6 6 8 6 8
D суммы матриц: 1 3 6 10

Данные успешно записаны в файл 'input1.txt'
```

matr_1 — Блокнот

1	0	7	0
0	2	0	8
7	0	3	6
0	8	6	4

matr_2 — Блокнот

7	8	0	6
8	9	0	0
0	0	3	0
6	0	0	4

input1 — Блокнот

8	8	7	6
8	11	0	8
7	0	6	6
6	8	6	8

Рис. 2 – Работоспособность программы сложения по схеме Дженнинга

Результат вычисления суммы различных матриц одинакового размера по схеме Дженнинга, где одна из матриц нулевая приведен на рис. 3.

The screenshot displays the output of a program in the Visual Studio console and three Notepad windows. The console window, titled 'Консоль отладки Microsoft Visual Studio', shows the following text:

```
Алгоритм Дженнинга - 1, кольцевая схема - 2: 1
Первая матрица
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0

AN первой матрицы: 0 0 0 0
D первой матрицы: 1 2 3 4

Вторая матрица
7 8 0 6
8 9 0 0
0 0 3 0
6 0 0 4

AN второй матрицы: 7 8 9 3 6 0 0 4
AN второй матрицы: 1 3 4 8

Сумма матриц:
7 8 0 6
8 9 0 0
0 0 3 0
6 0 0 4

AN суммы матриц: 7 8 9 3 6 0 0 4
D суммы матриц: 1 3 4 8

Данные успешно записаны в файл 'input1.txt'
```

The three Notepad windows are titled 'matr_1 — Блокнот', 'matr_2 — Блокнот', and 'input1 — Блокнот'. They contain the following data:

matr_1 — Блокнот:

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

matr_2 — Блокнот:

7	8	0	6
8	9	0	0
0	0	3	0
6	0	0	4

input1 — Блокнот:

7	8	0	6
8	9	0	0
0	0	3	0
6	0	0	4

Рис. 3 – Работоспособность программы сложение по схеме Дженнинга третий тест

Результат вычисления суммы нулевых матриц по схеме Дженнинга приведен на рис. 4.

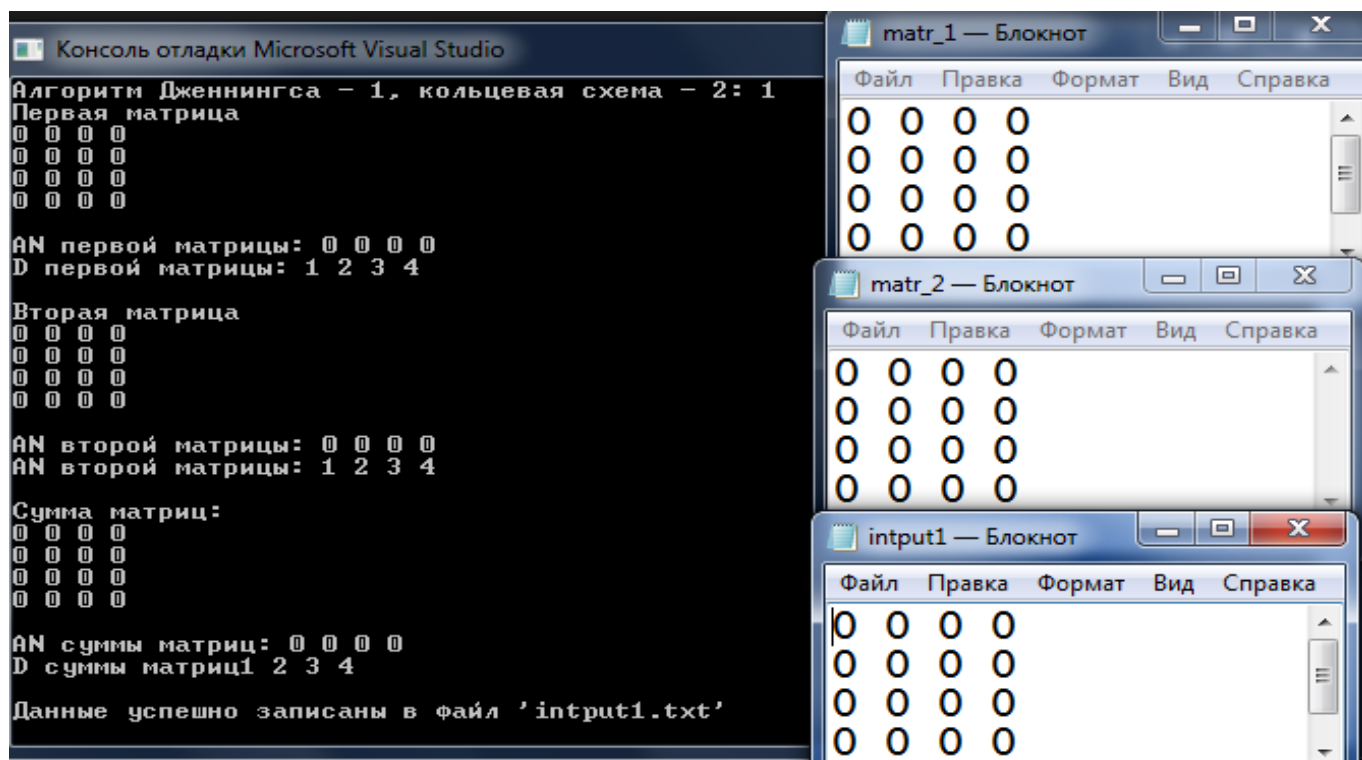


Рис. 4 – Работоспособность программы сложения по схеме Дженнинга

Результат вычисления суммы ненулевых матриц различного размера, используя кольцевую схему приведен на рис. 5.

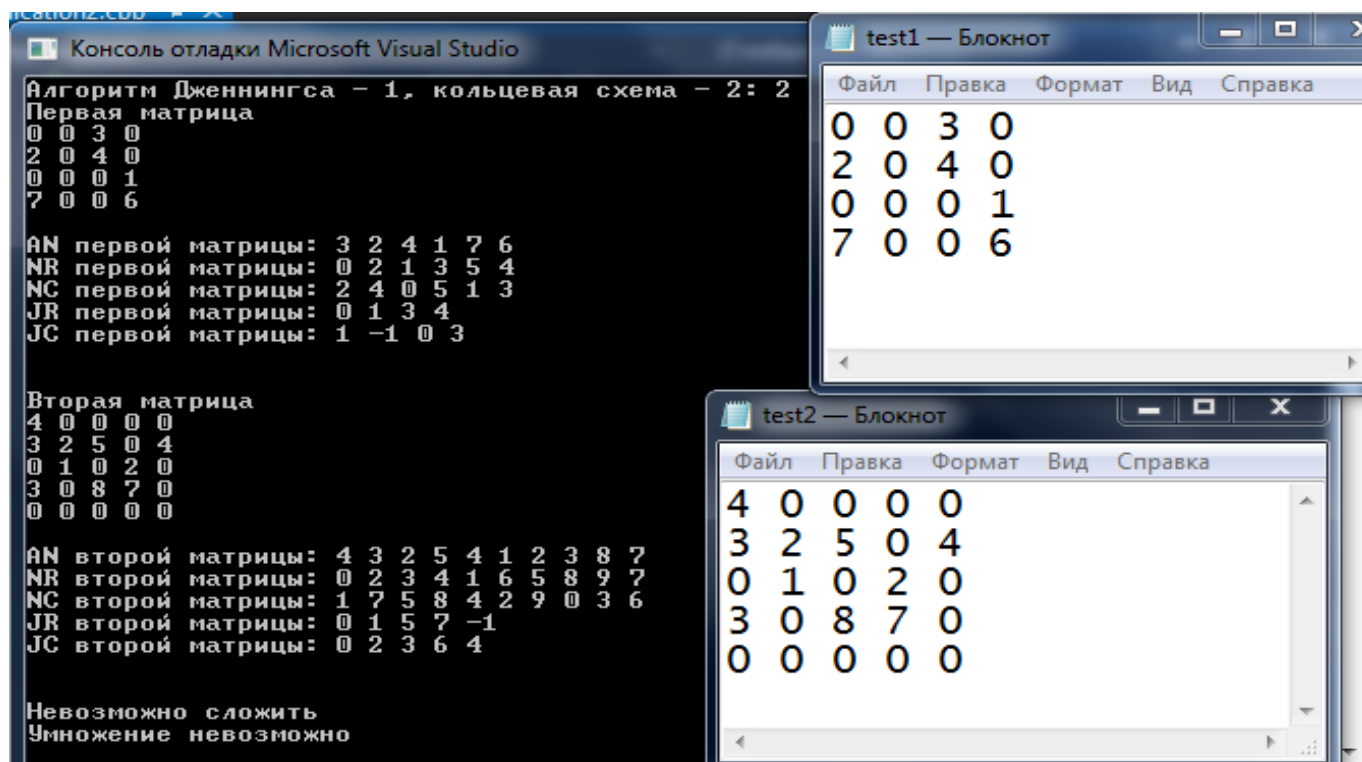


Рис. 5 – Работоспособность программы сложения по кольцевой схеме

Результат вычисления суммы нулевых матриц одинакового размера, используя кольцевую схему приведен на рис. 6.

```
Консоль отладки Microsoft Visual Studio
Алгоритм Дженнингса - 1, кольцевая схема - 2: 2
Первая матрица
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0

AN первой матрицы:
NR первой матрицы:
NC первой матрицы:
JR первой матрицы: -1 -1 -1 -1 -1
JC первой матрицы: -1 -1 -1 -1 -1

Вторая матрица
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0

AN второй матрицы:
NR второй матрицы:
NC второй матрицы:
JR второй матрицы: -1 -1 -1 -1 -1
JC второй матрицы: -1 -1 -1 -1 -1

Матрица суммы
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0

AN суммы матриц:
NR суммы матриц:
NC суммы матриц:
JR суммы матриц: -1 -1 -1 -1 -1
JC суммы матриц: -1 -1 -1 -1 -1

Данные успешно записаны в файл 'input2.txt'
```

test1 — Блокнот

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

test2 — Блокнот

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

input2 — Блокнот

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Рис. 6 – Работоспособность программы сложения по кольцевой схеме

Результат вычисления суммы ненулевых матриц одинакового размера, используя кольцевую схему приведен на рис. 7.

```

Консоль отладки Microsoft Visual Studio
Алгоритм Дженнингса - 1, кольцевая схема - 2: 2
Первая матрица
0 0 0 10 0 0 20
0 0 0 0 0 0 0
0 30 0 40 0 50 0
0 60 0 0 0 0 0
0 70 0 80 0 90 0
0 0 0 0 0 0 0

AN первой матрицы: 10 20 30 40 50 60 70 80 90
NR первой матрицы: 1 0 3 4 2 5 7 8 6
NC первой матрицы: 3 1 5 7 8 6 2 0 4
JR первой матрицы: 0 -1 2 5 6 -1
JC первой матрицы: -1 2 -1 0 -1 4 1

Вторая матрица
100 0 0 0 200 300 0
0 0 400 0 0 0 0
0 -30 0 10 20 -50 0
0 0 0 0 0 0 0
0 -70 0 -80 0 -90 0
0 0 10 0 0 0 0

AN второй матрицы: 100 200 300 400 -30 10 20 -50 -70 -80 -90
NR второй матрицы: 1 2 0 3 5 6 7 4 9 10 8 11
NC второй матрицы: 0 6 7 11 8 9 1 10 4 5 2 3
JR второй матрицы: 0 3 4 -1 8 11
JC второй матрицы: 0 4 3 5 1 2 -1

Матрица суммы
100 0 0 10 200 300 20
0 0 400 0 0 0 0
0 0 0 50 20 0 0
0 60 0 0 0 0 0
0 0 0 0 0 0 0
0 0 10 0 0 0 0

AN суммы матриц: 100 10 200 300 20 400 50 20 60 10
NR суммы матриц: 1 2 3 4 0 5 7 6 8 9
NC суммы матриц: 0 6 7 3 4 9 1 2 8 5
JR суммы матриц: 0 5 6 8 -1 9
JC суммы матриц: 0 8 5 1 2 3 4

Данные успешно записаны в файл 'input2.txt'

test1 — Блокнот
Файл  Правка  Формат  Вид  Справка
0 0 0 10 0 0 20
0 0 0 0 0 0 0
0 30 0 40 0 50 0
0 60 0 0 0 0 0
0 70 0 80 0 90 0
0 0 0 0 0 0 0

test2 — Блокнот
Файл  Правка  Формат  Вид  Справка
100 0 0 0 200 300 0
0 0 400 0 0 0 0
0 -30 0 10 20 -50 0
0 0 0 0 0 0 0
0 -70 0 -80 0 -90 0
0 0 10 0 0 0 0

input2 — Блокнот
Файл  Правка  Формат  Вид  Справка
100 0 0 10 200 300 20
0 0 400 0 0 0 0
0 0 0 50 20 0 0
0 60 0 0 0 0 0
0 0 0 0 0 0 0
0 0 10 0 0 0 0
  
```

Рис. 7 – Работоспособность программы сложения по кольцевой схеме

Результат умножения ненулевых матриц разного размера, используя кольцевую схему приведен на рис. 8.

```

Консоль отладки Microsoft Visual Studio
Алгоритм Дженнингса - 1, кольцевая схема - 2: 2
Первая матрица
0 0 0 10 0 0 20
0 0 0 0 0 0 0
0 30 0 40 0 50 0
0 60 0 0 0 0 0
0 70 0 80 0 90 0
0 0 0 0 0 0 0

AN первой матрицы: 10 20 30 40 50 60 70 80 90
NR первой матрицы: 1 0 3 4 2 5 7 8 6
NC первой матрицы: 3 1 5 7 8 6 2 0 4
JR первой матрицы: 0 -1 2 5 6 -1
JC первой матрицы: -1 2 -1 0 -1 4 1

Вторая матрица
100 0 0 0 200 300 0
0 0 400 0 0 0 0
0 -30 0 10 20 -50 0
0 0 0 0 0 0 0
0 -70 0 -80 0 -90 0
0 0 10 0 0 0 0

AN второй матрицы: 100 200 300 400 -30 10 20 -50 -70 -80 -90
NR второй матрицы: 1 2 0 3 5 6 7 4 9 10 8 11
NC второй матрицы: 0 6 7 11 8 9 1 10 4 5 2 3
JR второй матрицы: 0 3 4 -1 8 11
JC второй матрицы: 0 4 3 5 1 2 -1

Матрица суммы
100 0 0 10 200 300 20
0 0 400 0 0 0 0
0 0 0 50 20 0 0
0 60 0 0 0 0 0
0 0 0 0 0 0 0
0 0 10 0 0 0 0

AN суммы матриц: 100 10 200 300 20 400 50 20 60 10
NR суммы матриц: 1 2 3 4 0 5 7 6 8 9
NC суммы матриц: 0 6 7 3 4 9 1 2 8 5
JR суммы матриц: 0 5 6 8 -1 9
JC суммы матриц: 0 8 5 1 2 3 4

Данные успешно записаны в файл 'input2.txt'
  
```

Рис. 8 – Работоспособность программы умножения по кольцевой схеме

Результат умножения нулевых матриц разного размера, используя кольцевую схему приведен на рис. 9.

```
Консоль отладки Microsoft Visual Studio
Алгоритм Дженнингса - 1, кольцевая схема - 2: 2
Первая матрица
0 0 0
0 0 0
0 0 0
0 0 0

AN первой матрицы:
NR первой матрицы:
NC первой матрицы:
JR первой матрицы: -1 -1 -1 -1
JC первой матрицы: -1 -1 -1

Вторая матрица
0 0 0 0
0 0 0 0
0 0 0 0

AN второй матрицы:
NR второй матрицы:
NC второй матрицы:
JR второй матрицы: -1 -1 -1
JC второй матрицы: -1 -1 -1 -1

Невозможно сложить
Матрица произведения
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0

AN произведения матриц:
NR произведения матриц:
NC произведения матриц:
JR произведения матриц: -1 -1 -1 -1
JC произведения матриц: -1 -1 -1 -1
Данные успешно записаны в файл 'input3.txt'
```

```
test1 — Блокнот
Файл  Правка  Формат  Вид  Справка
0 0 0
0 0 0
0 0 0
0 0 0

test2 — Блокнот
Файл  Правка  Формат  Вид  Справка
0 0 0 0
0 0 0 0
0 0 0 0

input3 — Блокнот
Файл  Правка  Формат  Вид  Справка
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
```

Рис. 9 – Работоспособность программы умножения по кольцевой схеме

Результат умножения ненулевых матриц разного размера, используя кольцевую схему приведен на рис. 10.

The screenshot displays the output of a program in the Visual Studio console and three Notepad windows. The console shows the execution of the Jennings algorithm (1) and the ring scheme (2). It displays two input matrices, their row and column indices, and the resulting product matrix. The product matrix is shown in a 5x5 grid, with the first four rows containing non-zero values and the fifth row being all zeros. The Notepad windows show the same data in a more readable format.

```

Консоль отладки Microsoft Visual Studio
Алгоритм Дженнингса – 1, кольцевая схема – 2: 2
Первая матрица
1 0 0
0 0 0
0 0 3
4 0 0
0 2 0

AN первой матрицы: 1 3 4 2
NR первой матрицы: 0 1 2 3
NC первой матрицы: 2 1 0 3
JR первой матрицы: 0 -1 1 2 3
JC первой матрицы: 0 3 1

Вторая матрица
0 0 8 0 3
0 2 0 0 0
0 0 1 0 0

AN второй матрицы: 8 3 2 1
NR второй матрицы: 1 0 2 3
NC второй матрицы: 3 1 2 0
JR второй матрицы: 0 2 3
JC второй матрицы: -1 2 0 -1 1

Невозможно сложить
Матрица произведения
0 0 8 0 3
0 0 0 0 0
0 0 3 0 0
0 0 32 0 12
0 4 0 0 0

AN произведения матриц: 8 3 3 32 12 4
NR произведения матриц: 1 0 2 4 3 5
NC произведения матриц: 2 4 3 0 1 5
JR произведения матриц: 0 -1 2 3 5
JC произведения матриц: -1 5 0 -1 1
Данные успешно записаны в файл 'input3.txt'

test1 — Блокнот
Файл  Правка  Формат  Вид  Справка
1 0 0
0 0 0
0 0 3
4 0 0
0 2 0

test2 — Блокнот
Файл  Правка  Формат  Вид  Справка
0 0 8 0 3
0 2 0 0 0
0 0 1 0 0

input3 — Блокнот
Файл  Правка  Формат  Вид  Справка
0 0 8 0 3
0 0 0 0 0
0 0 3 0 0
0 0 32 0 12

```

Рис. 10 – Работоспособность программы умножения по кольцевой схеме

Оценим эффективность сжатого хранения матриц. Рассмотрим матрицу размера $n \times n$, где имеется k % ненулевых элементов, учитывая, что все элементы целые. Для хранения матрицы без сжатия потребуется $4 \cdot n^2$ байт памяти. Однако для хранения матриц, которые используются в схеме Дженнингса необходимо затратить $4 \cdot n$ байт памяти для массива D. Предположим, что в каждой строке находится m целых ненулевых элементов. Тогда на хранения массива AN необходимо затратить $4 \cdot m \cdot n$ байт памяти. В сумме матрицы AN и D занимают $4n \cdot (m + 1)$ байт памяти. Можно проследить зависимость необходимой памяти для хранения данных. При хранении всей матрицы имеем квадратичную зависимость, а при разреженном хранении — линейную. Отсюда нетрудно сделать вывод, что при значениях n стремящихся к бесконечности, выгоднее хранить разреженные матрицы.

Аналогичные размышления можно провести и с кольцевой схемой хранения. Пусть есть матрица $n \times n$ с m ненулевыми элементами. В ней используется 5 массивов, для хранения данных: AN, NC, NR по $4 \cdot m$ байт на массив. Также массивы JR и JC, которые занимают $4 \cdot n$ байт памяти каждый. Тогда в сумме эти массивы занимают $12m + 8n$ байт памяти. Получаем, что при n стремящемся к бесконечности выгоднее использовать кольцевую схему хранения разреженных матриц.

Заключение

Цель работы достигнута:

В результате выполнения лабораторной работы были выполнены все задачи.

1. Описаны две схемы сжатого хранения разреженных матриц — Дженнингса и кольцевая схема.
2. Разработаны алгоритмы сжатия, распаковки, сложения матриц для обеих схем и алгоритм умножения матриц для кольцевой схемы.
3. Реализованы разработанные алгоритмы.
4. Выполнено тестирование реализации разработанных алгоритмов.
5. Оценена эффективность сжатого хранения матриц в обеих схемах для матрицы действительных чисел с 5 % ненулевых элементов.

Согласно варианту, разработаны следующие два алгоритма.

1. Алгоритм сжатого хранения и распаковки матриц по схеме Дженнингса. Сжатые матрицы сложены и распакованы и результат был записан в файл.
2. Алгоритм сжатого хранения и распаковки матриц по схеме Рейнбольдта-Местеньи. Матрицы были сложены и умножены, запакованы и записаны в файл.

Анализ затрачиваемой на хранение матриц памяти показал, что при 5 — 10% ненулевых элементов и при бесконечно больших размерах матрицы выгоднее использовать сжатое хранение, нежели хранить полную матрицу.

Список литературы

1. Писсанецки, С. Технология разреженных матриц [Текст]: монография / С.Писсанецки.; пер. с англ. под ред. Х.Д.Икрамова. - М.: Мир, 1988. - 410 с.